

Python Workflow Engineering for AI Systems

Foundation Session 3

Mr. Shridhar Galande

SESSION AGENDA

What We'll Cover

- 1 Python Programming Fundamentals
- 2 Functions & Modularity
- 3 Classes & Object-Oriented Programming
- 4 File Handling
- 5 JSON & YAML Handling
- 6 Exception Handling
- 7 Logging
- 8 Configuration Management (.env + YAML)
- 9 Async Programming Basics
- 10 Type Hints & Clean Code (Black, Ruff)
- 11 Notebook vs Production Code
- 12 Reproducibility Practices
- 13 AI Project Structure (Integration)

— *Capstone + Wrap-up*

SESSION OVERVIEW

From Python Basics to Production AI Systems

This session builds the complete engineering toolkit behind real AI systems — starting with core Python, then layering on the practices that separate a working script from a production-ready system: modularity, OOP, robust error handling, logging, configuration, async I/O, type safety, and reproducibility.

Every topic is taught the same way: a plain-English explanation, a short worked example, and why it matters for AI/ML work — finishing with the standard project structure that ties every practice together, and a capstone that puts it all to use.

BY THE END OF THIS SESSION

- ✓ Write clean, modular, typed Python code
- ✓ Design simple classes for AI/ML components
- ✓ Handle files, JSON/YAML, and errors safely
- ✓ Build config- and .env-driven workflows
- ✓ Apply logging, async basics, and reproducibility

LEARNING OUTCOMES

What You'll Be Able to Do



Python Fundamentals

Write idiomatic Python using core data types and control flow



Classes & OOP

Model AI/ML components using classes, inheritance, and dataclasses



Exception Handling

Anticipate and handle failures without crashing a pipeline



Async Basics

Recognize when async I/O speeds up AI workflows



Functions & Modularity

Break code into reusable, well-organized functions and modules



File & Data Handling

Read/write files, JSON, and YAML safely and correctly



Logging & Config

Replace `print()` with logging and hardcoding with configs/.env



Clean Code & Reproducibility

Use type hints, Black, Ruff, and seeds for reliable, readable code

SECTION 01

Python Programming Fundamentals

Why Python for AI/ML?

Python is the default language of AI/ML — not by accident, but because of a specific combination of traits.

- Readable syntax — code reads close to plain English, easing collaboration
- Huge ecosystem — pandas, scikit-learn, PyTorch, TensorFlow, LangChain all live here
- Glue language — easily connects data pipelines, APIs, and infrastructure
- Fast to prototype, flexible enough to scale to production with the right practices

a taste of readability

```
scores = [91, 84, 77, 95]
average = sum(scores) / len(scores)
print(f"Average score: {average:.1f}")

# Average score: 86.8
```

Variables & Data Types

Every value in Python has a type. AI/ML code constantly moves data between these types — knowing them cold matters.

- int / float — numeric values (epochs, learning rate, accuracy)
- str — text (file paths, labels, log messages)
- bool — True/False (is_training, use_gpu)
- None — represents “no value yet”

example: model hyperparameters

```
learning_rate = 0.001      # float
epochs = 50                # int
model_name = "resnet50"    # str
use_gpu = True             # bool
checkpoint_path = None     # not set yet

print(type(learning_rate)) # <class 'float'>
```

Control Flow: if/else & Loops

Control flow lets code make decisions and repeat work — the backbone of any data processing step.

- if / elif / else — branch based on a condition
- for — repeat once per item in a collection
- while — repeat until a condition becomes false
- break / continue — exit early or skip an iteration

example: filtering a dataset

```
scores = [91, 42, 77, 38, 95]
passing = []

for score in scores:
    if score >= 60:
        passing.append(score)
    else:
        continue

print(passing)  # [91, 77, 95]
```


Core Data Structures

AI/ML pipelines constantly reshape data between these four containers.

- list — ordered, changeable: [91, 84, 77]
- tuple — ordered, unchangeable: ("train", "test")
- dict — key-value pairs: {"lr": 0.001, "epochs": 50}
- set — unique, unordered values: {"cat", "dog"}

example: a batch of records

```
batch = [  
    {"id": 1, "label": "cat"},  
    {"id": 2, "label": "dog"},  
]  
  
labels = {record["label"] for record in batch}  
print(labels)  # {'cat', 'dog'}
```

Strings & f-strings

Almost every AI workflow builds strings — file paths, prompts, log lines. f-strings are the clean, modern way.

- f"..." embeds variables directly inside a string
- Supports formatting — f"{value:.2f}" for 2 decimal places
- Cleaner than string concatenation with +
- Multiline strings with triple quotes for prompts/templates

example: a log message

```
epoch = 12
loss = 0.03421

message = f"Epoch {epoch}: loss={loss:.4f}"
print(message)

# Epoch 12: loss=0.0342
```

Quick Drill: Clean Up a Dataset

- 1 Given a list of dicts representing rows of data, loop through them
- 2 Use if/else to keep only rows where "score" $\geq$ 60
- 3 Build a set of unique "label" values seen in the kept rows
- 4 Print a clean f-string summary: how many rows kept, and which labels

Goal: Combine variables, control flow, data structures, and f-strings in one small script.

Section 1 Key Points



Python reads like English

Readable syntax + huge ecosystem make it the default AI/ML language.



Know your types

int, float, str, bool, None — data constantly moves between them.



Four core containers

list, tuple, dict, set each have a specific job — pick the right one.



f-strings for everything

Cleaner than concatenation for logs, paths, and prompts.

SECTION 02

Functions & Modularity

Why Functions Matter

Repeated logic scattered across a script is fragile — fix a bug in one place, forget the other three.

Before — repeated logic

```
acc1 = correct1 / total1
print(f"Model A: {acc1:.2%}")

acc2 = correct2 / total2
print(f"Model B: {acc2:.2%}")

acc3 = correct3 / total3
print(f"Model C: {acc3:.2%}")
```

After — one function

```
def report_accuracy(name, correct, total):
    acc = correct / total
    print(f"{name}: {acc:.2%}")

report_accuracy("Model A", correct1, total1)
report_accuracy("Model B", correct2, total2)
report_accuracy("Model C", correct3, total3)
```

Anatomy of a Function

Every function has the same shape: a name, parameters (with optional defaults), a body, and a return value.

- `def name(params):` — defines the function
- Default values (`epochs=10`) make parameters optional
- `return` sends a value back to the caller
- Docstrings document what a function does, for teammates and tools

example: a training function

```
def train_model(data, epochs=10, lr=0.001):  
    """Train a model and return final loss."""  
    loss = run_training(data, epochs, lr)  
    return loss  
  
final_loss = train_model(data, epochs=20)
```

*args and **kwargs

Sometimes you don't know in advance how many arguments a function needs — *args and **kwargs handle that flexibly.

- *args collects extra positional arguments into a tuple
- **kwargs collects extra keyword arguments into a dict
- Common in wrapper functions — logging, decorators, config overrides
- Use sparingly: explicit parameters are easier to read when possible

example: a flexible logging wrapper

```
def log_event(event_name, *args, **kwargs):  
    print(f"[{event_name}] args={args}  
    kwargs={kwargs}")  
  
log_event("training_start", 50, lr=0.001)  
# [training_start] args=(50,) kwargs={'lr': 0.001}
```


Organizing Code into Modules & Imports

- 1 Take three related functions (`load_data`, `train_model`, `evaluate`) written loosely in a notebook
- 2 Move them into a single file, `src/pipeline.py`
- 3 Add short docstrings to each function
- 4 In another file (or cell), import and use them: `from src.pipeline import train_model`
- 5 Confirm everything still runs the same as before — just better organized

Goal: See how splitting logic into functions and modules makes code reusable across files.

SECTION 03

Classes & Object- Oriented Programming

Why OOP for AI Systems?

Many AI/ML libraries (scikit-learn, PyTorch, Pydantic) are built entirely around classes — understanding OOP unlocks reading and extending them.

- Bundles data (attributes) and behavior (methods) together
- Models real-world concepts: a Dataset, a Model, a Pipeline
- Encourages reuse through inheritance instead of copy-pasting
- Every `model.fit(X, y)` call you've used is OOP in action

familiar OOP, from scikit-learn

```
from sklearn.ensemble import RandomForestClassifier

model = RandomForestClassifier() # an object
model.fit(X, y)                  # a method call
model.predict(X_new)             # another method
```

Classes, Objects & Attributes

A class is a blueprint. An object is a specific instance built from that blueprint, holding its own data (attributes).

- class Name: defines the blueprint
- `__init__` runs when a new object is created, setting up attributes
- `self` refers to the specific object being worked with
- Each object created from the class has its own independent attributes

example: a Dataset class

```
class Dataset:
    def __init__(self, name, rows):
        self.name = name
        self.rows = rows

train = Dataset("train", 1000)
print(train.name, train.rows)  # train 1000
```

Methods: Giving Objects Behavior

Methods are functions defined inside a class — they operate on the object's own data via `self`.

- Methods always take `self` as their first parameter
- They can read and update the object's attributes
- This is how a `Dataset` can clean or describe itself
- Keeps related data and logic together, instead of scattered functions

example: adding a method

```
class Dataset:
    def __init__(self, name, rows):
        self.name = name
        self.rows = rows

    def describe(self):
        return f"{self.name}: {self.rows} rows"

train = Dataset("train", 1000)
print(train.describe()) # train: 1000 rows
```

Inheritance & Polymorphism

Inheritance lets a class reuse and extend another class — avoiding duplicated logic across similar models.

- A subclass (child) inherits attributes/methods from a base class (parent)
- It can override a method to customize behavior — this is polymorphism
- Common pattern: BaseModel defines the shared interface
- Each specific model type implements its own version of train()

example: BaseModel -> RandomForestModel

```
class BaseModel:
    def train(self, X, y):
        raise NotImplementedError

class RandomForestModel(BaseModel):
    def train(self, X, y):
        self.model = RandomForestClassifier()
        self.model.fit(X, y)
        return self.model
```

@dataclass for Clean Configs

Writing `__init__` by hand for simple data-holding classes gets repetitive. `@dataclass` generates it for you.

- Add `@dataclass` above a class to auto-generate `__init__`
- Great for configs, records, and simple data containers
- Supports default values and type hints out of the box
- Far less boilerplate than a hand-written class

example: a training config

```
from dataclasses import dataclass

@dataclass
class TrainingConfig:
    learning_rate: float = 0.001
    epochs: int = 50
    model_name: str = "resnet50"

cfg = TrainingConfig(epochs=100)
print(cfg.epochs)  # 100
```

Build a Simple Model Class

- 1 Create a BaseModel class with a train(X, y) method that raises NotImplementedError
- 2 Create a RandomForestModel subclass that overrides train() using scikit-learn
- 3 Add an __init__ that accepts n_estimators as a parameter
- 4 Instantiate it and confirm .train(X, y) works end-to-end
- 5 Optional: define the same config as a @dataclass instead of loose variables

Goal: Practice classes, inheritance, and dataclasses together in one small, realistic example.

SECTION 04

File Handling

Reading & Writing Files

Almost every AI pipeline starts by reading a file and ends by writing one — predictions, logs, or checkpoints.

- `open(path, mode)` opens a file — "r" read, "w" write, "a" append
- The `with` statement auto-closes the file, even if an error occurs
- Always prefer `with open(...) as f:` over manual `open()/close()`
- Reading large files line-by-line avoids loading everything into memory

example: reading line by line

```
with open("data/raw_log.txt") as f:
    for line in f:
        if "ERROR" in line:
            print(line.strip())
```

pathlib for Cross-Platform Paths

String paths ("data\\file.csv") break across operating systems. pathlib.Path handles this correctly everywhere.

- from pathlib import Path — the modern way to handle paths
- Path("data") / "train.csv" joins paths safely on any OS
- .exists(), .mkdir(), .name, .suffix are all built in
- Used throughout our project/ structure from earlier sessions

example: building paths safely

```
from pathlib import Path

data_dir = Path("project") / "data"
data_dir.mkdir(parents=True, exist_ok=True)

file_path = data_dir / "train.csv"
print(file_path.exists()) # True or False
```

Working with CSV & Text

CSV is the most common data interchange format in ML workflows — pandas is the usual tool, but plain csv works too.

- `pandas.read_csv()` / `.to_csv()` for structured tabular data
- The built-in csv module for lightweight, dependency-free needs
- Always specify `index=False` when saving with pandas, unless you need it
- Writing predictions back to CSV is a common pipeline's final step

example: writing predictions

```
import pandas as pd

results = pd.DataFrame({
    "id": [1, 2, 3],
    "prediction": ["cat", "dog", "cat"],
})

results.to_csv("project/data/predictions.csv",
index=False)
```

Safely Read & Write a Dataset

- 1 Use pathlib to build a path to a sample CSV inside data/
- 2 Read it with pandas and print the number of rows
- 3 Add a new column (e.g., a dummy prediction)
- 4 Write the result to a new CSV inside data/, using Path for both input and output

Goal: Practice pathlib + file I/O together in a realistic read → transform → write flow.

SECTION 05

JSON & YAML Handling

JSON Basics

JSON is the universal format for structured data — API responses, metrics, and metadata all use it.

- `json.dump()` / `json.load()` write and read JSON files
- Maps directly to Python dicts and lists
- Great for saving results that need to be shared with other systems/languages
- Use `indent=2` when writing, so humans can read the file too

example: saving model metrics

```
import json

metrics = {"accuracy": 0.94, "f1_score": 0.91}

with open("project/logs/metrics.json", "w") as f:
    json.dump(metrics, f, indent=2)
```

YAML Basics

YAML is more human-friendly than JSON for configuration — no braces or quotes required, which is why config.yaml uses it.

- `yaml.safe_load()` / `yaml.dump()` from the PyYAML package
- Indentation (not braces) defines structure — be careful with spacing
- Ideal for configs/config.yaml — hyperparameters, paths, settings
- Always use `safe_load`, not `load`, to avoid executing arbitrary code

configs/config.yaml

```
model:
  type: random_forest
  n_estimators: 200
training:
  test_size: 0.2
  random_state: 42
```


Load & Save a Config

- 1 Write a Python dict with model settings (type, n_estimators, test_size)
- 2 Save it to configs/config.yaml using `yaml.dump()`
- 3 Read it back using `yaml.safe_load()` and print the values
- 4 Also save the same dict as metrics.json using `json.dump()` for comparison

Goal: See JSON and YAML side by side, and get comfortable converting between dicts and files.

SECTION 06

Exception Handling

Why Exception Handling Matters

Real-world data is messy — missing files, malformed rows, network timeouts. Unhandled errors crash the whole pipeline.

- An uncaught exception stops your script immediately
- In a long-running training job, one bad row shouldn't kill the whole run
- Handling errors well means failing gracefully, with a clear message
- The goal isn't to hide errors — it's to respond to them deliberately

without handling: this crashes

```
df = pd.read_csv("data/does_not_exist.csv")

# Traceback (most recent call last):
# FileNotFoundError: [Errno 2] No such file
# ... entire script stops here
```

try / except / else / finally

Python's exception-handling block lets you attempt risky code and respond specifically when it fails.

- try: the code that might fail
- except `SpecificError`: runs only if that error type occurs
- else: runs only if no exception occurred
- finally: always runs — cleanup code belongs here

example: safe data loading

```
try:
    df = pd.read_csv("data/train.csv")
except FileNotFoundError:
    print("Data file missing – check data/
    folder")
    df = None
else:
    print(f"Loaded {len(df)} rows")
finally:
    print("Data loading step complete")
```

Custom Exceptions

Built-in exceptions don't always describe your specific problem. Custom exceptions make errors self-explanatory.

- Define a new exception by subclassing Exception
- Raise it with `raise YourError("message")` when a business rule is violated
- Makes except blocks elsewhere in the codebase more precise
- Especially useful for data validation in ML pipelines

example: a custom validation error

```
class DataValidationError(Exception):  
    """Raised when input data fails a quality  
    check."""  
    pass  
  
def validate(df):  
    if df.isnull().values.any():  
        raise DataValidationError("Dataset  
contains null values")
```

Build a Robust `load_data` Function

- 1 Write `load_data(path)` that reads a CSV with pandas
- 2 Wrap it in `try/except` to catch `FileNotFoundError` with a clear message
- 3 Raise a custom `DataValidationError` if the loaded `DataFrame` is empty
- 4 Add a `finally` block that logs “`load_data` finished” regardless of outcome

Goal: Practice `try/except/else/finally` and a custom exception in one realistic function.

SECTION 07

Logging

Why Logging Beats print()

print() disappears the moment your terminal closes. Logging keeps a permanent, filterable, timestamped record.

print() — disappears, unstructured

```
print("Training started")
print("Epoch 1 done")
print("Something went wrong!")

# No timestamps, no severity levels,
# lost when the terminal closes
```

logging — persistent, structured

```
import logging
logging.basicConfig(
    filename="logs/train.log",
    level=logging.INFO
)
logging.info("Training started")
logging.error("Something went wrong!")
```


Log Levels & Setup

Logging has severity levels so you can filter noise from what actually matters, without changing your code.

- DEBUG < INFO < WARNING < ERROR < CRITICAL, in increasing severity
- `basicConfig(level=...)` sets the minimum level that gets recorded
- Write to a file (`filename=...`) so logs survive after the run ends
- Include `%(asctime)s` in the format for automatic timestamps

example: full setup

```
import logging

logging.basicConfig(
    filename="logs/train.log",
    level=logging.INFO,
    format="%(asctime)s %(levelname)s\n%(message)s",
)

logging.info("Training started")
logging.warning("Learning rate seems high")
```

Add Logging to a Training Script

- 1 Configure logging to write to logs/train.log at INFO level
- 2 Log an info message when training starts and when it completes
- 3 Log a warning if accuracy is below a threshold (e.g., 0.7)
- 4 Open logs/train.log afterward and confirm the timestamped entries

Goal: Replace ad-hoc print() statements with a proper logging setup.

SECTION 08

Configuration Management

Stop Hardcoding, Start Configuring

Hardcoded values mean every change requires editing code. Config-driven workflows separate settings from logic.

Before — hardcoded

```
model = RandomForestClassifier(  
    n_estimators=200  
)  
test_size = 0.2  
path = "data/train.csv"  
  
# change anything -> edit the script
```

After — config-driven

```
import yaml  
  
with open("configs/config.yaml") as f:  
    config = yaml.safe_load(f)  
  
model = RandomForestClassifier(  
    n_estimators=config["model"]["n_estimators"]  
)
```

config.yaml vs .env

Two different jobs, two different files — never mix them.

	configs/config.yaml	.env
Holds	Non-secret parameters (model type, hyperparameters, paths)	Secrets (API keys, tokens, credentials)
Committed to git?	Yes	Never — only .env.example is committed
Loaded with	yaml.safe_load()	python-dotenv → os.getenv()

Secrets & .env Files

API keys, tokens, and credentials never belong in code or config.yaml — that's what .env is for.

- Store secrets as KEY=VALUE lines in a .env file
- Load them with python-dotenv: load_dotenv() then os.getenv("KEY")
- .env is always added to .gitignore — never committed
- Share a .env.example with the same variable names and placeholder values

.env (never committed)

```
API_KEY=sk-live-a1b2c3d4e5

# loading it in Python
from dotenv import load_dotenv
import os

load_dotenv()
api_key = os.getenv("API_KEY")
```

Build a Config- and .env-Driven Script

- 1 Create configs/config.yaml with model settings
- 2 Create .env with a dummy API_KEY, and .env.example as its template
- 3 Write a script that loads both: `yaml.safe_load()` and `load_dotenv()`
- 4 Print the config values and a masked version of the API key
- 5 Add .env (but not .env.example) to .gitignore

Goal: Practice cleanly separating non-secret settings from secrets in one working script.

SECTION 09

Async Programming Basics

Why Async Matters for AI Workflows

Many AI workflows spend most of their time waiting — for an API response, a file download, a database query. Async lets you overlap that waiting instead of doing it one task at a time.

- Calling 3 slow APIs one-by-one takes the sum of all 3 wait times
- Calling them concurrently with async takes roughly the longest single wait
- Especially relevant for LLM API calls, web scraping, and I/O-heavy pipelines
- Async doesn't speed up CPU-heavy work (like model training) — only I/O waiting

the problem: 3 sequential API calls

```
results = []
for prompt in prompts:
    results.append(call_llm_api(prompt))

# each call waits ~2s -> 3 prompts = ~6s total
```

Sync vs Async: The Event Loop

In synchronous code, each line waits for the previous one to finish. Async code can pause a waiting task and work on another, using an event loop to juggle them.

- Synchronous: task A finishes completely, then task B starts
- Asynchronous: task A starts, hits a wait, task B starts during that wait
- The event loop is what manages switching between paused/ready tasks
- This is cooperative — tasks must explicitly say “I'm waiting” with `await`

conceptually

```
# Sync:    |--A--| |--B--| |--C--|      (6s total)

# Async:   |--A--|
#          |--B--|
#          |--C--|      (~2s total)
```

async / await Syntax

Python's async support is built around two keywords: `async def` to define a coroutine, and `await` to pause until it's done.

- `async def fetch()`: defines a coroutine (an awaitable function)
- `await` pauses this coroutine until the awaited call completes
- `asyncio.run(main())` is the standard way to start the event loop
- `asyncio.gather(...)` runs multiple coroutines concurrently

example: fetching a prediction

```
import asyncio

async def fetch_prediction(prompt):
    await asyncio.sleep(1) # simulates a network call
    return f"prediction for: {prompt}"

async def main():
    result = await fetch_prediction("hello")
    print(result)

asyncio.run(main())
```

Simulated Concurrent API Calls

- 1 Write an async function `fetch_prediction(prompt)` that awaits `asyncio.sleep(1)` then returns a string
- 2 Write a sync version of the loop calling it 3 times — time it with `time.time()`
- 3 Write an async version using `asyncio.gather()` to call all 3 concurrently — time it
- 4 Compare the two run times and discuss why they differ

Goal: See, hands-on, why async concurrency helps for I/O-bound AI workflow tasks.

SECTION 10

Type Hints & Clean-Code Practices

Why Type Hints

Type hints document what a function expects and returns — catching mistakes before code even runs, and helping tools autocomplete correctly.

Before — no hints

```
def train_model(data, epochs, lr):  
    ...  
    return model  
  
# What is 'data'? A DataFrame? A list?  
# Is 'epochs' an int or a string?
```

After — typed

```
def train_model(  
    data: pd.DataFrame,  
    epochs: int,  
    lr: float,  
) -> RandomForestClassifier:  
    ...  
    return model
```

Common Typing Patterns

The typing module covers most real-world cases you'll need for AI/ML code.

- `Optional[str]` — a value that might be `None`
- `List[float]`, `Dict[str, int]` — typed collections
- `Union[int, float]` — accepts either type
- Type hints are not enforced at runtime — they're for readability & tooling (mypy, IDEs)

example: a typed data-loader

```
from typing import Optional, List, Dict

def load_data(
    path: str,
    columns: Optional[List[str]] = None,
) -> Dict[str, list]:
    ...
    return {"rows": [], "columns": columns or []}
```

Naming & Docstrings (PEP 8)

Clean code is mostly about consistent, boring conventions — PEP 8 is Python's official style guide.

- snake_case for variables and functions, PascalCase for classes
- Descriptive names beat comments: n_estimators, not n or x
- A one-line docstring explains what a function does, not how
- Consistency across a codebase matters more than any single rule

example: PEP 8 in practice

```
class RandomForestModel:          # PascalCase
    def train_and_evaluate(       # snake_case
        self, X_train, y_train
    ):
        """Train the model and return its
accuracy."""
        ...
```


Black: The Uncompromising Formatter

Black automatically reformats your code to a single, consistent style — no debates about spacing, quotes, or line length.

Before black .

```
def train(data,epochs=10,lr =0.001):  
    model=RandomForestClassifier(  
        n_estimators = 200)  
    model.fit(data['X'],data[ 'y'])  
    return model
```

After black .

```
def train(data, epochs=10, lr=0.001):  
    model =  
    RandomForestClassifier(n_estimators=200)  
    model.fit(data["X"], data["y"])  
    return model  
  
# consistent spacing, quotes, indentation –  
# automatically, every time
```

Ruff: Fast, Modern Linting

Ruff checks your code for bugs, unused imports, and style issues — far faster than older linters, and increasingly the industry default.

- `ruff check .` scans a project and reports issues with file + line numbers
- Catches real bugs: unused variables, undefined names, unreachable code
- `ruff check --fix .` auto-fixes many issues (like unused imports) directly
- Runs in milliseconds, even on large codebases — easy to run on every save

```
$ ruff check .
```

```
src/train.py:3:8: F401 'pandas as pd' imported  
      but unused  
src/train.py:12:5: F841 local variable 'temp'  
      is assigned to but never used  
  
Found 2 errors.
```

Format & Lint a Sample File

- 1 Take a provided messy, untyped `src/train.py` file
- 2 Add type hints to each function signature
- 3 Run `pip install black ruff`, then `black .` to auto-format the file
- 4 Run `ruff check .` and fix any issues it reports (try `ruff check --fix .` too)
- 5 Compare the file before and after — note what changed automatically vs. manually

Goal: Get hands-on with the exact Black + Ruff workflow used on real production codebases.

SECTION 11

Notebook vs Production Code

When to Use Which



Notebooks

- Best for exploration & prototyping
- See output immediately, cell by cell
- Great for visualizing data & results
- Hard to test, version, or reuse



Scripts (.py)

- Best for production & repeatable runs
- Runs top-to-bottom, no hidden state
- Testable, schedulable, automatable
- Reusable as importable modules

Refactor Demo

The same logic, before and after graduating from a notebook cell to a production-ready function.

Notebook cell

```
df = pd.read_csv("data.csv")
df = df.dropna()
X = df.drop("label", axis=1)
y = df["label"]
model = RandomForestClassifier()
model.fit(X, y)
```

src/train.py

```
def load_data(path: str) -> pd.DataFrame:
    return pd.read_csv(path).dropna()

def train(X, y) -> RandomForestClassifier:
    model = RandomForestClassifier()
    model.fit(X, y)
    return model
```

Notebook → Production Checklist



Extract logic into functions

Split notebook cells into small, named, testable functions.



Move settings to config.yaml

No hardcoded paths or hyperparameters left in the script.



Add logging, not print()

Every run leaves a traceable, timestamped record.



Add type hints + run Black/Ruff

Formatted, linted, and self-documenting before it ships.

SECTION 12

Reproducibility Practices

Why Reproducibility Matters

“It worked on my machine” is the most expensive sentence in AI/ML. Reproducibility means the same code + data reliably gives the same result.

- Randomness in ML (data splits, weight init) means results can differ run to run
- Different library versions can silently change model behavior
- Without reproducibility, debugging and comparing experiments becomes guesswork
- Reproducibility is what makes results trustworthy and shareable

same code, different results?

```
model = RandomForestClassifier()  
model.fit(X_train, y_train)  
print(model.score(X_test, y_test))  
  
# Run 1: 0.87    Run 2: 0.84    Run 3: 0.89  
# - no random_state set!
```

Seeds, Versioning & Environment Pinning

Three practices get you most of the way to reproducible results.

- Fix `random_state` / seeds everywhere randomness is used
- Pin exact library versions in `requirements.txt`
- Track which `config.yaml` produced which result
- Log the seed and key settings alongside every run's output

example: pinned & seeded

```
# requirements.txt
scikit-learn==1.5.1
pandas==2.2.2

# train.py
model = RandomForestClassifier(random_state=42)
model.fit(X_train, y_train)  # same result, every
run
```

Reproducibility Meets Project Structure



configs/

Every run's exact settings are saved and versionable.



logs/

Records which seed, config, and result belong together.



requirements.txt

Pinned versions mean the environment itself is reproducible.



.env (not versioned)

Keeps secrets out of the reproducible, shareable parts of a project.

Make a Training Run Reproducible

- 1 Take a training script without any `random_state` set
- 2 Run it three times and confirm the accuracy differs each time
- 3 Add `random_state=42` everywhere randomness occurs (split, model init)
- 4 Re-run three times and confirm the accuracy is now identical
- 5 Pin the exact library versions used into `requirements.txt`

Goal: Directly observe non-reproducible vs. reproducible results, side by side.

SECTION 13

AI Project Structure

Why Structure Matters, Now That You've Seen the Pieces

You've now learned functions, classes, file handling, JSON/YAML, exceptions, logging, configs, async, type hints, and reproducibility. Professional teams don't scatter these across a folder randomly — there's a standard layout.

- A predictable structure means any teammate can open a project and know where things live
- Each practice from this session maps to exactly one place in that structure
- This is the same “notebook chaos → organized project” idea, now with the full toolkit behind it
- It's also exactly what the capstone will build inside, next

the standard layout

```
project/  
├─ data/  
├─ models/  
├─ notebooks/  
├─ src/  
├─ configs/  
├─ logs/  
├─ .env  
└─ .env.example
```

Folder-by-Folder, Mapped to What We Learned

Folder	What Belongs Here	Section It Connects To
<code>data/</code>	Raw & processed datasets, read/written via file handling	Sec. 4 — File Handling
<code>models/</code>	Serialized model artifacts produced by training classes	Sec. 3 — Classes & OOP
<code>notebooks/</code>	Exploration — later refactored into <code>src/</code>	Sec. 11 — Notebook vs Production
<code>src/</code>	Functions, classes, typed & linted with Black/Ruff	Sec. 2, 3, 10
<code>configs/</code>	<code>config.yaml</code> — non-secret parameters	Sec. 8 — Configuration Mgmt
<code>logs/</code>	Logging output; also where run metadata for reproducibility lives	Sec. 7, 12
<code>.env</code>	Secrets, loaded with <code>python-dotenv</code>	Sec. 8 — Configuration Mgmt

Naming Conventions & Best Practices



Use lowercase + underscores

train_model.py, not TrainModel.PY — consistent, PEP-8-aligned names.



Never hardcode file paths

Keep paths in configs/ so the same code runs on any machine.



One notebook, one purpose

Split “explore data” and “train model” into separate notebooks.



Keep data/, models/, .env out of git

Use .gitignore — large files and secrets don't belong in version control.



Type-hint and lint everything in src/

Run black . and ruff check . before committing.



Pin your environment

requirements.txt with exact versions, for reproducibility.

Scaffold Your Own Project

- 1 Create the project/ root with the six standard sub-folders
- 2 Add a sample dataset into data/, and a config.yaml into configs/
- 3 Add .env and .env.example, and a .gitignore excluding data/, models/, logs/, .env
- 4 Move a function you wrote earlier in this session into src/, with type hints
- 5 Run `black .` and `ruff check .` on src/ to confirm it's clean

Goal: End with a working, organized project skeleton — ready for the capstone.

Common Project-Organization Mistakes



Everything in one notebook

No separation between data loading, training, and evaluation code.



Hardcoded absolute paths

Breaks the moment it runs on another machine.



Secrets committed to git

API keys in config.yaml or code end up permanently in git history.



No logs, no seeds, no trace

Failures and inconsistent results become impossible to debug.

CAPSTONE

Bringing It All Together

Refactor a Messy Legacy Script

- 1 Start from a provided `messy_pipeline.py` — one giant untyped, unstructured script
- 2 Scaffold the standard project/ structure and move the script's logic into `src/`
- 3 Split the logic into typed functions and/or a small class hierarchy
- 4 Move hardcoded values into `configs/config.yaml` and `.env`
- 5 Wrap risky steps in `try/except`, and add logging instead of `print()`
- 6 Set `random_state` for reproducibility, and run `black . + ruff check .` on the result

Goal: Touch every topic from this session — fundamentals, functions, OOP, files, JSON/YAML, exceptions, logging, config, type hints, clean code, and reproducibility — inside one real project structure.

RECAP

Key Takeaways



Fundamentals are the foundation

Clean variables, control flow, and data structures underlie everything else.



Functions & classes organize logic

Modularity and OOP turn scripts into maintainable systems.



Robustness isn't optional

Exception handling, logging, and configs make code trustworthy in production.



Structure + reproducibility tie it together

The standard project layout, type hints, Black/Ruff, and seeds make results shareable and repeatable.



Questions & Discussion

Thank you — let's build well-engineered AI systems together.